Universidad Abierta Interamericana
Facultad de Tecnología Informática
Ingeniería en Sistemas Informáticos

TRABAJO FINAL DE INGENIERÍA

Decisiones de arquitectura

16 registros de decisión (ADR), con su contexto y sus consecuencias

Alumno: Pettini, Patricio Ezequiel
Legajo: B00072710-T1
Sede: Sede Norte — Boulogne
Docentes: Romero, Martín Miguel · Weingand, Gastón Matías

Versión 1.0 · 23/10/2026

Índice

Por qué se registran 6
Cómo se relacionan con el resto de la documentación 7
ADR 0001 — Registrar las decisiones de arquitectura 8
Contexto 8
Decisión 8
Consecuencias 9
ADR 0002 — C# / ASP.NET Core en lugar de Java / Spring Boot 10
Contexto 10
Decisión 10
Alternativas consideradas 11
Consecuencias 11
ADR 0003 — Dos bases de datos separadas: negocio y servicio 13
Contexto 13
Decisión 13
Alternativas consideradas 14
Consecuencias 14
ADR 0004 — ADO.NET con DAO explícito en lugar de Entity Framework Core 16
Contexto 16
Decisión 16
Alternativas consideradas 17
Consecuencias 17
ADR 0005 — Multi-organización desde el modelo de datos 19
Contexto 19
Decisión 19
Alternativas consideradas 20
Consecuencias 20
ADR 0006 — Proveedor de IA conmutable con asignación de respaldo 21
Contexto 21

Decisión 21
Alternativas consideradas 22
Consecuencias 22
ADR 0007 — Reglas predictivas parametrizadas, resueltas con estrategias 24
Contexto 24
Decisión 24
Alternativas consideradas 25
Consecuencias 25
ADR 0008 — Nivel de exigencia en carga y seguridad 27
Contexto 27
Decisión 27
1. Análisis estático (SAST) 27
2. Pruebas de carga 28
3. Perfilado de memoria 28
Alternativas consideradas 29
Consecuencias 29
Revisión del 23/10: lo que el ADR decía y lo que el proyecto tenía 30
ADR 0009 — Patrones de resiliencia: circuit breaker y bulkhead 32
Contexto 32
Decisión 32
Circuit Breaker sobre IProveedorIA 32
Bulkhead para los procesos en segundo plano 33
Alternativas consideradas 33
Consecuencias 34
Precisión de la implementación (02/09/2026) 34
ADR 0010 — Cifrado reversible, separado del hash de contraseñas 36
Contexto 36
Decisión 37
Cómo queda la clave de API 37

Consecuencias 38
Alternativas descartadas 38
ADR 0011 — El motor predictivo se dispara solo 40
Contexto 40
Decisión 40
Por hecho — ReevaluarPorEvento(idEquipo) 40
Por tiempo — BarridoPredictivo 41
Consecuencias 41
Alternativas descartadas 42
ADR 0012 — La máquina de estados de la incidencia vive en la BLL 43
Contexto 43
Decisión 43
Las transiciones 43
En un solo lugar 44
Tres reglas que van con la decisión 44
Dónde termina la validación y dónde empieza la guía 44
Consecuencias 45
Alternativas descartadas 45
ADR 0013 — Reportes en PDF generados en el servidor 46
Contexto 46
Decisión 46
Por qué en el servidor y no en el navegador 46
Por qué QuestPDF 46
El generador no sabe qué reporta 47
Los reportes leen los datos que ya tiene el sistema 47
Detalles de composición que son decisiones 47
El PDF no se guarda 47
IReporteBusiness va aparte 47
Consecuencias 48

Alternativas descartadas 48
ADR 0014 — Gestión de identidad propia, sin IAM externo 49
Contexto 49
Decisión 49
El motivo principal no es técnico 49
Lo que además pesa 50
Lo que se pierde, y hay que decirlo 50
Consecuencias 51
Alternativas descartadas 51
ADR 0015 — Métricas en formato Prometheus, sin stack de observabilidad propio 52
Contexto 52
Decisión 52
Qué se expone 53
Se usa una biblioteca para el formato y no se escribe a mano 53
El endpoint no es público 53
Alternativas consideradas 54
Consecuencias 54
0016 · Alta de usuarios con contraseña temporal y sesión sin patentes 56
Contexto 56
Alternativas consideradas 56
Decisión 57
Lo que se pierde, y hay que decirlo 57
Consecuencias 57
Relación con el resto 58
Fuentes 59

Por qué se registran

Registro de las decisiones de arquitectura significativas de PredictIT, en formato Nygard. El criterio para escribir uno está en el ADR 0001.

Un ADR aprobado no se edita. Si la decisión cambia, se escribe uno nuevo que lo reemplaza y el anterior pasa a estado *Reemplazado*.

#	Decisión	Estado	Impacto en el documento
0001	Registrar las decisiones de arquitectura	Aprobado	—
0002	C# / ASP.NET Core en lugar de Java / Spring Boot	Aprobado	Contradice 10.2.5 · D-01
0003	Dos bases de datos separadas: negocio y servicio	Aprobado	—
0004	ADO.NET con DAO explícito en lugar de EF Core	Aprobado	—
0005	Multi-organización desde el modelo de datos	Aprobado	Amplía 10.4.7 · D-02
0006	Proveedor de IA conmutable con asignación de respaldo	Aprobado	Amplía 1.7 / 6.4.1 / 10.2.3 · D-03, D-04
0007	Reglas predictivas parametrizadas, resueltas con estrategias	Aprobado	—
0008	Nivel de exigencia en carga y seguridad	Aprobado	Amplía 10.2.2 RNF · D-08
0009	Patrones de resiliencia: circuit breaker y bulkhead	Aprobado	Amplía 10.3 · D-09
0010	Cifrado reversible, separado del hash de contraseñas	Aprobado	Amplía 10.2.2 RNF-04 · D-16
0011	El motor predictivo se dispara solo	Aprobado	Precisa 10.2.2 RF-11 · D-17
0012	La máquina de estados de la incidencia vive en la BLL	Aprobado	Amplía 10.2.3 CU-008 · D-20
0013	Reportes en PDF generados en el servidor	Aprobado	Precisa 10.2.2 RF-14 · D-21
0014	Gestión de identidad propia, sin IAM externo	Aprobado	Amplía 10.3.3 · D-31

#	Decisión	Estado	Impacto en el documento
0015	Métricas en formato Prometheus, sin stack propio	Aprobado	Hace verificable 10.2.2 RNF-05 · D-53
0016	Alta de usuarios con contraseña temporal y sesión sin patentes	Aprobado	Amplía 10.3.3 · precisa RF-02 · D-98

Cómo se relacionan con el resto de la documentación

- Cuando una decisión obliga a modificar el documento del TFI, queda anotada en `docs/CAMBIOS-DOCUMENTO.md` con la sección exacta.
- Cuando la decisión se tomó consultando a una herramienta de IA, la consulta y su validación quedan en `docs/trazabilidad/PROMPTS-CRITICOS.md`.

ADR 0001 — Registrar las decisiones de arquitectura

Estado: Aprobado

Fecha: 2026-09-02

Contexto

El desarrollo de PredictIT se apoya de manera intensiva en asistencia por IA. Eso acelera la escritura de código, pero traslada el peso de la ingeniería desde *escribir* hacia *decidir y validar*: la pregunta relevante en la defensa no va a ser cómo se escribió una clase, sino por qué el sistema está construido así y qué se descartó en el camino.

El documento del TFI describe el sistema en su estado final, no el razonamiento que llevó hasta ahí. Sin un registro explícito, las decisiones quedan implícitas en el código y se pierden: dentro de dos meses nadie —ni el propio autor— puede reconstruir por qué el acceso a datos es ADO.NET y no Entity Framework, ni qué alternativas se evaluaron.

Decisión

Se registran las decisiones de arquitectura significativas como **Architecture Decision Records** en formato Nygard, en `docs/adr/`, numerados secuencialmente y versionados junto al código.

Una decisión es significativa —y por lo tanto lleva ADR— cuando cumple al menos una de estas condiciones:

- Es costosa de revertir una vez escrito el código que depende de ella.
- Contradice lo que dice el documento del TFI.
- Se eligió entre alternativas reales, no por falta de opciones.
- Determina cómo se cumple un requerimiento no funcional.

Cada ADR lleva Título, Estado, Contexto, Decisión y Consecuencias, separando en estas últimas lo positivo de lo negativo y los riesgos asumidos. Un ADR aprobado no se edita: si la decisión cambia, se escribe uno nuevo que lo reemplaza y el anterior pasa a estado *Reemplazado*.

Los ADR se enlazan desde `docs/CAMBIOS-DOCUMENTO.md` cuando la decisión obliga a modificar el documento del TFI, y desde `docs/trazabilidad/PROMPTS-CRITICOS.md` cuando la decisión se tomó consultando a una herramienta de IA.

Consecuencias

Positivas

- Las decisiones quedan defendibles con su contexto y sus alternativas, que es exactamente lo que se evalúa en la defensa oral.
- El capítulo de arquitectura del documento se redacta a partir de los ADR en vez de reconstruirse de memoria al final.
- Un ADR obliga a escribir las consecuencias negativas, lo que fuerza a mirar el costo de cada decisión antes de tomarla y no después.

Negativas y riesgos

- Cuesta disciplina: un ADR escrito una semana tarde ya perdió el contexto que lo hacía valioso.
- Hay riesgo de sobredocumentar. El criterio de significatividad de arriba existe para eso: si una decisión se revierte cambiando dos archivos, no lleva ADR.
- El formato puede degradarse en trámite si se escribe para cumplir. Un ADR sin alternativas descartadas no aporta nada.

ADR 0002 — C# / ASP.NET Core en lugar de Java / Spring Boot

Estado: Aprobado

Fecha: 2026-09-02

Contradice: documento del TFI, sección 10.2.5 (ver `docs/CAMBIOS-DOCUMENTO.md`, D-01)

Contexto

La sección 10.2.5 del documento fija el entorno del sistema en Spring Boot / Java / MySQL / IIS 10. Esa definición se escribió antes de empezar a construir y no estaba atada a ninguna restricción real del proyecto: no hay infraestructura preexistente, ni integraciones que impongan una plataforma, ni cliente que la exija.

Al mismo tiempo hay dos condicionantes concretos que sí pesan:

1. **La cátedra evalúa la separación en capas** (DOMAIN / DAO / BLL / SERVICE) y los patrones que la sostienen. Existe un proyecto de referencia propio —el Trabajo de Diploma de 3.º año— que implementa exactamente ese modelo en C# con ADO.NET, con `IGenericDao<T>`, `IObjectMapper<T>`, Abstract Factory, Unit of Work y el esquema de permisos Usuario / Familia / Patente ya resuelto.
2. **C# es el lenguaje visto a lo largo de la carrera.** El TFI se defiende oralmente y hay que poder explicar cualquier línea del sistema bajo pregunta.

El costo de elegir Java sería construir desde cero un andamiaje que ya está resuelto y probado, en un lenguaje con el que el autor tiene menos recorrido, sin ninguna ventaja compensatoria para este dominio: no hay volumen de datos, concurrencia ni requisitos de despliegue que diferencien seriamente a una plataforma de la otra en la escala de este sistema (500 activos y 5000 incidencias, según RNF-04).

Decisión

El sistema se desarrolla en **C# sobre .NET 9**, con **ASP.NET Core Web API** como backend y **React 19 + TypeScript** como frontend, contra **SQL Server 2022**.

La solución replica la estructura del proyecto de referencia llevada a web: `PredictIT.Domain`, `PredictIT.DAO`, `PredictIT.BLL`, `PredictIT.Service`, `PredictIT.Api` y `PredictIT.Tests`, con las dependencias cableadas de modo que sólo puedan ir hacia abajo.

Se actualiza la sección 10.2.5 del documento y los diagramas de componentes y despliegue en consecuencia.

Alternativas consideradas

Opción	Por qué se descartó
Java / Spring Boot / MySQL, como dice el documento	Reescribir desde cero un andamiaje ya resuelto, en el lenguaje con menos recorrido del autor, sin ventaja técnica para este dominio.
ASP.NET Core MVC con vistas Razor	Una sola solución y menos piezas móviles, pero el módulo del cliente y el panel de administración tienen necesidades de interacción distintas, y el frontend separado permite tratarlos como dos aplicaciones sobre la misma API.
Node.js / TypeScript en todo el stack	Sin instalación previa y con el mayor recorrido práctico del autor, pero se aleja del lenguaje de la carrera y del modelo de capas que evalúa la cátedra.

Consecuencias

Positivas

- Reutiliza un modelo de capas ya validado académicamente, con sus patrones identificables uno por uno en el código.
- El autor puede defender cualquier parte del sistema en el lenguaje que conoce.
- SQL Server habilita `BACKUP` / `RESTORE DATABASE` nativo, con lo cual el Req. Arq. 003 se cumple sin construir un mecanismo propio.

Negativas y riesgos

- Obliga a modificar el documento en varios puntos: entorno, diagramas de componentes y de despliegue, y manual de instalación.
- Ata el sistema a un motor de base de datos comercial. Para el alcance del TFI se cubre con la edición Developer en contenedor, pero un despliegue productivo real tendría un costo de licencia que Java + MySQL no tendría.
- El frontend separado agrega una pieza más al despliegue y al manual de instalación que una solución monolítica no tendría.

ADR 0003 — Dos bases de datos separadas: negocio y servicio

Estado: Aprobado

Fecha: 2026-09-02

Contexto

El DER del documento (10.4.7) ya está dibujado como dos diagramas: *DER Negocio* —equipos, incidencias, mantenimientos, alertas— y *DER Servicio* —usuarios, patentes, familias, bitácora—. Queda por decidir si esa separación conceptual se traduce en dos bases físicas o en dos grupos de tablas dentro de una sola.

La tensión es concreta. Una sola base permite claves foráneas entre `Incidencia.id_tecnico` y `Usuario`, integridad referencial garantizada por el motor, y consultas con `JOIN` directo. Dos bases obligan a resolver esas referencias por convención y a hacer dos consultas donde podría haber una.

A favor de separar pesan tres cosas:

- El Req. Arq. 002 exige que la bitácora sea inmutable y auditable. Una base de seguridad con su propio ciclo de respaldo y sus propios permisos hace esa exigencia estructural en vez de declarativa.
- El sistema es multi-organización (ADR 0005). Los datos de negocio de un cliente y las credenciales de acceso tienen ciclos de vida y sensibilidades distintas.
- La cátedra evalúa la separación de responsabilidades, y una base de servicio reutilizable entre sistemas es la forma más literal de demostrarla. El proyecto de referencia de 3.º año hace exactamente esto con `FastFoodDB` y `ServicesPP`.

Decisión

Se usan **dos bases de datos con cadenas de conexión separadas**:

- `PredictIT_Negocio` — organizaciones, ubicaciones, equipos, incidencias, clasificaciones, recomendaciones de asignación, mantenimientos, reglas, alertas y evaluaciones de riesgo.
- `PredictIT_Servicio` — usuarios, roles, familias, patentes, bitácora, tipos de evento, idiomas, traducciones y respaldos.

Ningún DAO de negocio puede alcanzar la base de servicio ni al revés: cada capa de acceso recibe su propia cadena y no conoce la otra. Las referencias cruzadas —`Equipo.id_responsable`, `Incidencia.id_tecnico`, `Mantenimiento.id_tecnico`— son **referencias lógicas por GUID**, documentadas como tales en el esquema, sin clave foránea.

La integridad de esas referencias se valida en la BLL antes de persistir, no en el motor.

Alternativas consideradas

Opción	Por qué se descartó
Una sola base con esquemas `negocio` y `seguridad`	Conserva las claves foráneas y simplifica todo, pero la separación pasa a ser una convención de nombres que nada impide violar, y se pierde el argumento de respaldo y permisos independientes.
Dos bases con *synonyms* o consultas de tres partes	Recupera el `JOIN` a costa de perforar exactamente el aislamiento que motiva la decisión.

Consecuencias

Positivas

- La separación es real y verificable: un DAO de negocio no tiene forma de leer una contraseña ni de escribir en la bitácora.
- Cada base se respalda y se restaura por separado, y la de servicio puede reutilizarse en otro sistema sin arrastrar el dominio.
- La inmutabilidad de la bitácora se garantiza donde corresponde —un trigger `INSTEAD OF UPDATE, DELETE` en la base de servicio— y no depende de que la aplicación se porte bien.

Negativas y riesgos

- **No hay integridad referencial entre bases.** Un usuario borrado deja incidencias apuntando a un GUID inexistente. Se mitiga con baja lógica (`Usuario.activo`), nunca física, y con validación en la BLL.
- Toda pantalla que muestre "el técnico de la incidencia" necesita dos consultas y una composición en la BLL. Es el costo recurrente de la decisión.
- Las pruebas de integración tienen que preparar y limpiar dos bases, y una transacción no puede abarcar las dos sin transacción distribuida —que deliberadamente no se usa—. Las operaciones que tocan ambas (registrar una incidencia y asentar la bitácora) no son atómicas: se resuelve asentando la bitácora **después** del commit de negocio, aceptando que un fallo entre ambos pasos pierde el registro de auditoría pero nunca el dato de negocio.

ADR 0004 — ADO.NET con DAO explícito en lugar de Entity Framework Core

Estado: Aprobado

Fecha: 2026-09-02

Contexto

Elegido C# (ADR 0002), queda decidir cómo se accede a los datos. La opción por defecto de la plataforma es Entity Framework Core: menos código, migraciones automáticas, LINQ tipado y materialización de objetos resuelta.

El problema es qué queda de la capa DAO si se usa EF. Un `DbContext` **es** la capa de acceso a datos: escribir un `IEquipoDao` que envuelve un `DbSet<Equipo>` produce una capa que no hace nada más que delegar, y la separación que la cátedra evalúa pasa a ser decorativa. El patrón Unit of Work tampoco tendría sentido propio: EF ya lo implementa internamente en el `SaveChanges`.

En sentido contrario pesa que el volumen del sistema es chico —RNF-04 habla de 500 activos y 5000 incidencias— y que la mayoría de las consultas son CRUD sobre una tabla. Es decir: el argumento de rendimiento a favor de SQL a mano no aplica acá, y el de productividad a favor de EF sí, pero contra el objetivo académico.

Hay además un factor de continuidad: el proyecto de referencia de 3.º año usa ADO.NET con un `SqlHelper` estático, `IObjectMapper<T>` para mapear el `SqlDataReader`, y `SqlTransactRepository` como Unit of Work. Ese código está probado y es directamente portable.

Decisión

El acceso a datos se implementa con **ADO.NET** (`Microsoft.Data.SqlClient`), SQL parametrizado con `SqlParameter` y mapeo explícito, siguiendo la estructura del proyecto de referencia:

- `IGenericDao<T>` como contrato base, con los DAO específicos extendiéndolo.
- `IObjectMapper<T>` por entidad, que traduce `SqlDataReader` a objeto de dominio.
- `SqlHelper` para la base de negocio y `SqlHelperSeguridad` para la de servicio, cada uno con su cadena (ADR 0003).

- `IUnitOfWork` / `SqlTransactRepository` para las operaciones que abarcan varias tablas dentro de una transacción.
- `FactoryDao` como Abstract Factory, único punto donde se instancian los DAO.

Todo parámetro va por `SqlParameter`. No se concatena entrada de usuario en una sentencia bajo ninguna circunstancia; es la mitigación de inyección SQL y se verifica por análisis estático (ver ADR 0008).

El esquema se versiona como **scripts SQL idempotentes** en `db/`, no con migraciones generadas: el script es el artefacto que se entrega y se lee en la defensa, y tiene que ser legible.

Alternativas consideradas

Opción	Por qué se descartó
EF Core con `DbContext`	Vacía de contenido la capa DAO y el Unit of Work, que es justamente lo que se evalúa.
Dapper	Buen punto medio —SQL a la vista, mapeo automático— pero el mapeo automático es precisamente el `IObjectMapper<T>` que hay que mostrar escrito.
EF Core sólo para los catálogos y ADO.NET para el resto	Dos modelos mentales de acceso a datos en el mismo sistema, sin ganancia real.

Consecuencias

Positivas

- Cada patrón que la cátedra evalúa está escrito y es señalable en el código, no delegado a un framework.
- Control total sobre el SQL que se ejecuta, incluidas las consultas del motor predictivo, que agrupan por ventana temporal y no se expresan cómodamente en LINQ.
- Los scripts de `db/` son el esquema, legibles sin herramientas.

Negativas y riesgos

- **Mucho más código repetitivo.** Cada entidad necesita su DAO, su mapper y sus cuatro sentencias. Es el costo aceptado y hay que sostener la consistencia entre ellos.
- Un cambio de columna se propaga a mano: script, entidad, mapper y cada sentencia que la nombre. Sin compilador que avise. Se mitiga con pruebas de integración de DAO que recorren el CRUD completo de cada entidad contra la base real.

- El riesgo de inyección SQL pasa a depender de la disciplina del desarrollador en lugar de estar eliminado por construcción. Mitigado por revisión y SAST.

ADR 0005 — Multi-organización desde el modelo de datos

Estado: Aprobado

Fecha: 2026-09-02

Amplía: documento del TFI, sección 10.4.7 (ver `docs/CAMBIOS-DOCUMENTO.md`, D-02)

Contexto

El plan de negocio del TFI define tres segmentos, y el **Segmento C** —proveedores tercerizados de soporte informático— es explícitamente el canal estratégico: cada proveedor introduce el sistema en varias PyMEs a la vez. La sección 5.3.6 le pone precio a eso: *Plan Partner, ARS 27.900 por cliente gestionado*, con acceso a un "portal multi-cliente".

El DER del documento, sin embargo, no tiene entidad `Organizacion`. Las entidades cuelgan unas de otras y el modelo asume implícitamente una sola empresa por instalación. Con ese modelo, el plan Partner sólo se puede implementar desplegando una instancia y una base por cliente, que es exactamente lo que un proveedor con cinco clientes chicos no va a operar.

La decisión hay que tomarla ahora: agregar aislamiento por organización una vez que existen los DAO, la BLL y la API significa tocar cada consulta del sistema.

Decisión

Se incorpora `Organizacion` como **raíz del modelo de negocio**. Toda tabla de negocio lleva `id_organizacion`: `Ubicacion`, `Equipo`, `Incidencia`, `Mantenimiento`, `ReglaAlerta`, `AlertaPredictiva`, `EvaluacionRiesgo`, `TecnicoEspecialidad` y `ConfiguracionAsignacion`.

El aislamiento se aplica **en la capa DAO**, tomando la organización del contexto de sesión, no del pedido HTTP. Un controlador no puede pedir datos de otra organización porque no tiene forma de nombrarla: el identificador no viaja en la petición, se resuelve del token.

Los usuarios llevan `id_organizacion` en la base de servicio como referencia lógica (ADR 0003). El perfil **Partner** es la excepción: tiene `id_organizacion` nulo y sus organizaciones habilitadas viven en `Usuario_Organizacion`. Cambiar de organización activa requiere la patente `ORGANIZACION_CAMBIAR`, que sólo tiene ese rol, y queda asentado en bitácora.

Los catálogos verdaderamente globales —tipos de equipo, estados, prioridades, categorías, especialidades— se comparten entre organizaciones. Las reglas predictivas y la configuración de asignación **no**: cada cliente calibra su propio motor.

Alternativas consideradas

Opción	Por qué se descartó
Una instalación y una base por cliente	Aislamiento perfecto y modelo más simple, pero convierte al plan Partner en algo inoperable: cinco clientes son cinco despliegues, cinco respaldos y cinco actualizaciones.
Un esquema de SQL Server por organización	Aislamiento fuerte sin multiplicar instancias, pero el alta de un cliente pasa a ser una operación DDL y el Partner no puede consultar de forma consolidada.
Dejarlo como evolución futura	Es la opción que más se paga: agregar `id_organizacion` con el sistema construido obliga a revisar cada consulta, cada test y cada endpoint.

Consecuencias

Positivas

- El plan Partner del capítulo 5 pasa a ser implementable, y el sistema puede demostrar en la defensa el escenario que sostiene el canal de venta.
- El aislamiento en el DAO hace que olvidarse del filtro sea difícil: no hay una consulta "sin organización" que un desarrollador pueda escribir por descuido.
- Permite demostrar el aislamiento con datos: el seed carga dos organizaciones justamente para poder probar que una no ve a la otra.

Negativas y riesgos

- **Es el riesgo de seguridad más serio del sistema.** Una consulta que se olvide el filtro filtra datos entre clientes. Se mitiga con el filtro obligatorio en el DAO, pruebas de integración específicas de aislamiento por cada entidad, y revisión dirigida a eso en el análisis estático.
- Todas las claves únicas pasan a ser compuestas con la organización: el código `PC-ADM-014` puede existir en dos clientes distintos.
- Complica el modelo y el DER respecto de lo que dice el documento, que hay que actualizar.
- El Partner introduce un estado de sesión adicional —la organización activa— que es una fuente clásica de errores si se cachea mal.

ADR 0006 — Proveedor de IA conmutable con asignación de respaldo

Estado: Aprobado

Fecha: 2026-09-02

Amplía: documento del TFI, secciones 1.7, 6.4.1 y 10.2.3 (ver `docs/CAMBIOS-DOCUMENTO.md`, D-03 y D-04)

Contexto

El documento define la asignación automática de incidencias mediante un servicio externo de IA (RF-08, CU-007) y su comportamiento ante falla (CP-14). El alcance se amplía además con la clasificación del texto libre del solicitante (D-03/D-04).

Eso introduce en el camino crítico del sistema una dependencia que el sistema **no controla**: un servicio de terceros, por red, con latencia variable, que puede estar caído, devolver algo malformado o —el caso más incómodo— responder correctamente pero con el nombre de un técnico que no existe.

Hay además dos restricciones prácticas:

1. **No hay crédito de API disponible.** Un sistema que sólo funciona con una clave cargada no se puede demostrar en la defensa si ese día la cuenta no tiene saldo o la red del aula falla.
2. **CP-14 exige poder provocar la caída del servicio** para verificar la asignación de respaldo. Depender de un servicio real hace ese caso de prueba no determinístico, y un caso de prueba que a veces pasa no prueba nada.

Decisión

La integración se define detrás de una interfaz de dominio, **`IProveedorIA`**, con dos operaciones: clasificar una incidencia y recomendar un técnico. Se implementa dos veces:

- **`ProveedorIAClaude`** — consume la API de Anthropic con salida JSON estructurada.
- **`ProveedorIASimulado`** — heurística determinística, sin red, y con la capacidad de simular fallas y demoras a pedido.

La implementación activa se elige por configuración **por organización** (`ConfiguracionAsignacion.proveedor_ia`), no por variable de entorno global: es el cliente el que decide si sus datos salen o no hacia un tercero. El valor por defecto es `SIMULADO`.

Sobre la llamada real se aplican tres protecciones, en este orden:

1. **Timeout** configurable, 15 segundos por defecto.
2. **Validación de la respuesta** contra el conjunto de técnicos que efectivamente existen, están activos y pertenecen a la organización. Una recomendación que nombra a alguien fuera de ese conjunto se descarta como si el servicio hubiera fallado.
3. **Asignación de respaldo**: técnico con menor carga, `tipo_asignacion = RESPALDO`, incidencia marcada `pendiente_revision`, y evento en bitácora con el motivo.

El resultado de la IA es siempre **una sugerencia editable**, nunca un hecho consumado: la clasificación se guarda en `ClasificacionIncidencia` separada de los campos reales de la incidencia, y la recomendación en `RecomendacionAsignacion` junto con el contexto exacto que se envió.

Ningún flujo del sistema queda bloqueado esperando a la IA. Si el servicio no está, el sistema funciona con heurísticas y lo dice.

Alternativas consideradas

Opción	Por qué se descartó
Llamar a la API directamente desde la BLL	Ata el sistema a un proveedor, hace las pruebas dependientes de la red y vuelve CP-14 no reproducible.
Simular en las pruebas con un *mock* y usar sólo la implementación real en producción	El *mock* prueba el código de prueba. Acá el modo simulado es también un modo de operación legítimo: es como se demuestra el sistema sin crédito de API.
Entrenar un modelo propio sobre el historial	Fuera del alcance declarado en 6.4.1, y sin volumen de datos que lo justifique: una PyME con 50 equipos no genera un conjunto de entrenamiento.
Cola asincrónica: registrar la incidencia y asignar después	Elimina la latencia del camino crítico, pero agrega una pieza de infraestructura desproporcionada y rompe la expectativa de CU-007, que asigna al registrar.

Consecuencias

Positivas

- El sistema es demostrable siempre, con o sin conexión y con o sin crédito.
- CP-14 pasa a ser un caso de prueba determinístico y automatizable: se fuerza la falla en el proveedor simulado y se verifica el respaldo.
- Cambiar de proveedor de IA es escribir una clase nueva, sin tocar la BLL.
- `RecomendacionAsignacion` guarda el contexto enviado, lo que permite auditar después qué se le mandó a un tercero y por qué recomendó lo que recomendó.

Negativas y riesgos

- **Hay dos caminos de código y sólo uno se ejercita en la demostración.** Si sólo se prueba el simulado, la implementación real puede estar rota sin que nadie se entere. Se mitiga con un conjunto de pruebas de contrato que ambas implementaciones deben satisfacer, y una prueba de integración contra el servicio real marcada para ejecución manual.
- El modo simulado puede dar una impresión inflada de la capacidad del sistema si no se aclara en la defensa cuál está activo. La pantalla de Integración de IA muestra el proveedor en uso, y ése es el punto donde se aclara.
- Guardar el contexto enviado implica almacenar datos sobre la carga de trabajo de personas identificables. Es información laboral sensible: se guarda porque hace auditable la decisión, y se acota con las casillas de la pantalla de configuración, donde el cliente elige qué se envía.
- La validación de la respuesta contra los técnicos reales es imprescindible: sin ella, una respuesta plausible pero inventada asignaría la incidencia a nadie.

ADR 0007 — Reglas predictivas parametrizadas, resueltas con estrategias

Estado: Aprobado

Fecha: 2026-09-02

Contexto

El motor predictivo es el diferencial del producto (1.7, 10.2.1) y RF-11 lo define como reglas **configurables** sobre el historial. El documento nombra cinco condiciones: recurrencia de fallas, mantenimiento vencido, acumulación de incidencias, antigüedad del equipo y garantía por vencer.

"Configurable" admite grados muy distintos, y hay que elegir uno:

- Que el administrador ajuste **parámetros** de reglas predefinidas (cuántas fallas, en cuántos días, cuánto pesa).
- Que pueda **escribir condiciones nuevas** en algún lenguaje de expresiones.
- Que las reglas estén fijas en el código y sólo se activen o desactiven.

El usuario objetivo es un responsable administrativo no técnico. Un editor de expresiones es una funcionalidad que nadie de ese perfil va a usar y que introduce un evaluador de expresiones —es decir, ejecución de entrada de usuario— en el sistema. Reglas fijas en código, en el otro extremo, contradicen RF-11 y dejan al producto sin el argumento de que cada cliente calibra su motor.

Hay además una tensión de modelado: cada tipo de regla necesita parámetros distintos. "Recurrencia" necesita cantidad y ventana temporal; "antigüedad" necesita años; "garantía" necesita días de aviso. Modelar eso con columnas produce una tabla mayormente nula, y con una tabla de parámetros clave-valor produce consultas incómodas para algo que siempre se lee entero.

Decisión

Las reglas se guardan en `ReglaAlerta` con un campo `tipo` que discrimina la estrategia y un campo `condicion` con **los parámetros en JSON**:

```
tipo = 'RECURRENCIA_FALLAS'
condicion = {"minIncidencias": 3, "ventanaDias": 30}
```

```
peso = 30
```

Cada tipo se resuelve con una **estrategia** (patrón Strategy) que sabe leer sus propios parámetros y devuelve un aporte al score más el motivo textual que lo explica. El `EvaluadorReglas` recorre las reglas activas de la organización, selecciona la estrategia por `tipo` y compone el resultado.

El score final es la **suma ponderada de los aportes**, acotada a 0–100, y se traduce a nivel según umbrales: bajo 0–39, medio 40–69, alto 70–100. Cada evaluación se persiste completa en `EvaluacionRiesgo`, con el detalle por regla en JSON.

El `tipo` está restringido por `CHECK` en la base al conjunto de estrategias implementadas: no se puede guardar una regla que el sistema no sepa evaluar.

Agregar un tipo de regla nuevo es escribir una estrategia y agregar su valor al `CHECK`. **No** es una operación que pueda hacer el usuario final.

Alternativas consideradas

Opción	Por qué se descartó
Lenguaje de expresiones editable por el usuario	Nadie del perfil objetivo lo usaría, y mete evaluación de entrada de usuario en el sistema: superficie de ataque sin beneficio.
Una columna por parámetro en `ReglaAlerta`	Tabla mayormente nula, y cada tipo de regla nuevo es una migración de esquema.
Tabla `ParametroRegla` clave-valor	Normaliza bien pero se lee siempre entera de todos modos, y pierde el tipado que igual hay que reconstruir al parsear.
Reglas fijas en código, sólo activables	Contradice RF-11 y elimina el argumento comercial de calibración por cliente.

Consecuencias

Positivas

- El motor es determinístico y se prueba sin base de datos ni red: cada estrategia es una función de (historial, parámetros) a (puntos, motivo).
- La pantalla de Reglas predictivas puede ofrecer un preview real —"con estos valores se dispararía en 4 equipos"— porque evaluar es barato.
- Persistir `EvaluacionRiesgo` con el detalle permite responder la pregunta que va a aparecer en la defensa: por qué este equipo estaba en rojo hace dos semanas y hoy no.

- El motor explica sus decisiones. Cada aporte al score viene con el dato concreto que lo generó, y es el desglose que muestra la pantalla de detalle de equipo.

Negativas y riesgos

- El JSON de `condicion` no está validado por el motor de base: una regla con parámetros mal formados sólo falla al evaluarse. Se mitiga validando el JSON contra el esquema de cada tipo en la BLL al guardar, no al usar.
- La suma ponderada acotada a 100 hace que, con muchas reglas activas y pesos altos, todo tienda a saturar en riesgo alto y el score pierda capacidad de discriminar. La pantalla de reglas muestra la suma de pesos para que sea visible.
- El score es una construcción, no una probabilidad de falla. Presentarlo como "86 % de probabilidad de romperse" sería falso; se presenta como score de riesgo y así hay que sostenerlo en la defensa.
- Cambiar una regla cambia el score histórico si se reevalúa. Por eso `EvaluacionRiesgo` guarda la foto: las evaluaciones pasadas no se recalculan.

ADR 0008 — Nivel de exigencia en carga y seguridad

Estado: Aprobado

Fecha: 2026-09-02

Amplía: documento del TFI, sección 10.2.2 — RNF (ver `docs/CAMBIOS-DOCUMENTO.md`, D-08)

Contexto

El desarrollo se apoya en asistencia por IA, y eso cambia el perfil de riesgo del proyecto, no sólo su velocidad. El código asistido tiende a ser sintácticamente correcto y plausible, lo que desactiva la señal de alarma más útil que tiene un desarrollador: que algo *se vea mal*. Los defectos que sobreviven a esa revisión son justamente los que no se ven —una consulta sin filtro de organización, un `SqlCommand` armado por concatenación en un método que quedó fuera del patrón, un `SqlDataReader` sin `using`, una dependencia con vulnerabilidad conocida que el asistente sugirió por costumbre—.

La consigna del trabajo lo plantea directamente: dado el incremento de velocidad de codificación, el nivel de exigencia en pruebas de estrés, perfilado de memoria y análisis estático debe ser **superior al estándar tradicional**.

Los RNF actuales del documento no alcanzan para eso. RNF-04 fija un objetivo de rendimiento (menos de 2 segundos con 500 activos y 5000 incidencias) pero no dice cómo se verifica, y RNF-02 habla de seguridad en términos de control de acceso, sin mencionar análisis de código ni dependencias.

Decisión

Se incorporan al proyecto tres controles obligatorios, ejecutables por comando y verificados antes de cada entrega formal. Ninguno es opcional ni queda "para si hay tiempo".

1. Análisis estático (SAST)

- **Analizadores de .NET en modo estricto**: `TreatWarningsAsErrors=true` en `Directory.Build.props`. Una advertencia rompe la compilación; no hay advertencias toleradas. (El texto original pedía además `AnalysisMode=All`; ver la revisión del 23/10 al final.)
- **`dotnet list package --vulnerable --include-transitive`** en cada entrega. Una dependencia con vulnerabilidad conocida bloquea la entrega.

- **Reglas dirigidas a los riesgos propios de este sistema**, más allá del conjunto genérico:
- CA2100 — SQL construido por concatenación. Es el riesgo directo de haber elegido ADO.NET (ADR 0004).
- CA2000 / CA2213 — `SqlConnection` y `SqlDataReader` sin liberar.
- Ausencia de filtro por organización en un DAO. No hay analizador que lo detecte: se cubre con revisión dirigida y con pruebas de aislamiento (ADR 0005).
- Secretos fuera del repositorio: la clave de API se guarda cifrada en base y las cadenas de conexión reales van por configuración de entorno.

2. Pruebas de carga

Se define un perfil de carga que refleja el escenario de RNF-04 y se verifica contra él, no contra una corrida a ojo:

Escenario	Volumen	Objetivo
Consulta de inventario con filtros	500 activos	p95 < 2 s
Historial técnico de un equipo	5000 incidencias en base	p95 < 2 s
Dashboard	agregados sobre el total	p95 < 2 s
Evaluación del motor predictivo	parque completo	tiempo total acotado y medido
Registro de incidencia con IA simulada	20 usuarios concurrentes	p95 < 2 s

Se mide sobre una base cargada con volumen sintético, y se registra el resultado de cada corrida para poder mostrar la evolución, no sólo el último número. (El texto original nombraba NBomber; se midió con un script propio, ver la revisión del 23/10.)

El escenario que más interesa no es el más rápido sino el más frágil: el motor predictivo sobre el parque completo, porque su costo crece con el historial y el historial sólo crece.

3. Perfilado de memoria

Con las herramientas del propio SDK, sin dependencias adicionales: `dotnet-counters` para observar el comportamiento bajo carga sostenida, `dotnet-gcdump` para comparar el montón antes y después de una corrida larga, y `dotnet-trace` cuando haga falta ubicar un punto caliente.

Se busca específicamente lo que este sistema puede tener: conexiones o lectores no liberados durante la evaluación del parque completo, y acumulación en el caché de permisos del `SeguridadService`.

Alternativas consideradas

Opción	Por qué se descartó
SonarQube o CodeQL	Cobertura superior, pero agregan infraestructura y tiempo de configuración desproporcionados para el alcance del TFI. Los analizadores del SDK cubren los riesgos identificados.
JMeter o k6 para la carga	Herramientas más difundidas, pero quedan fuera de la solución. NBomber vive en el proyecto de pruebas y se corre con `dotnet test`.
Dejar la carga y el perfilado para el cierre	Es cuando ya no hay margen para actuar sobre lo que se encuentre. Se corren desde que existe la API.

Consecuencias

Positivas

- Convierte "el nivel de exigencia debe ser superior" en comandos concretos que se corren y devuelven un número.
- `TreatWarningsAsErrors` obliga a resolver los avisos en el momento, con el contexto todavía fresco.
- Los resultados de carga alimentan directamente la verificación de RNF-04, que hoy es una afirmación sin evidencia.

Negativas y riesgos

- `TreatWarningsAsErrors` es incómodo: rompe compilaciones por cosas menores y tienta a suprimir reglas. Toda supresión va con comentario que la justifique, y las supresiones se revisan en cada entrega.
- Generar volumen sintético realista lleva trabajo y el resultado nunca es igual a datos reales: 5000 incidencias distribuidas uniformemente no estresan lo mismo que 5000 concentradas en pocos equipos. El generador produce distribución sesgada a propósito.
- Ninguno de estos controles detecta el defecto más peligroso del sistema —una consulta sin filtro de organización—. Eso depende de las pruebas de aislamiento. Conviene no confundir la existencia del control con la ausencia del riesgo.

Revisión del 23/10: lo que el ADR decía y lo que el proyecto tenía

Revisando la segunda entrega de tecnología contra el código aparecieron tres diferencias entre esta decisión y lo construido. Se corrigen acá y no se borran del texto de arriba, porque el registro de una decisión también registra cuándo dejó de describir el sistema.

`AnalysisMode=All` y `TreatWarningsAsErrors` nunca estuvieron en `Directory.Build.props`. El historial del repositorio no tiene ni un commit que los agregue. El CI compilaba con `-warnaserror`, así que las advertencias del conjunto por defecto sí cortaban, pero sólo ahí, y las tres reglas dirigidas no estaban activas: por defecto no corren. Se midió qué costaba cumplir lo escrito:

Modo	Advertencias	Qué son
Por defecto	0	—
`Recommended`	unas 400	Nombres con guion bajo —las 560 de CA1707 son los nombres de las pruebas, y son así a propósito—, cultura en el formateo, sobrecargas
`All`	unas 730	Lo anterior, más `ConfigureAwait` (CA2007), que en ASP.NET Core no aplica, y validación de argumentos públicos

Activar `All` obligaba a suprimir cientos de reglas en bloque, que es tener el control escrito sin tenerlo. Se activó lo que la decisión necesitaba de verdad: `TreatWarningsAsErrors` en todas las máquinas, y CA2100, CA2000 y CA2213 elevadas a error en `backend/analizadores.globalconfig`. Al activarlas aparecieron tres usos de SQL con texto variable, los tres correctos por construcción —el ayudante que ejecuta el SQL de los DAO, el nombre de base del respaldo validado contra una lista blanca y una consulta constante de una prueba—, y quedaron suprimidos uno por uno con su motivo al lado. Se comprobó que el control corta: un `SqlCommand` armado por concatenación y otro sin liberar, agregados a propósito en un DAO, rompen la compilación.

Había una dependencia con una vulnerabilidad de severidad alta. `System.Security.Cryptography.Pkcs` 9.0.13, que trae `Microsoft.Data.SqlClient`, quedó alcanzada por la CVE-2026-84008 —denegación de servicio— publicada el 13/10, un mes después de fijar las versiones. El job de dependencias del CI avisa y, como dice la revisión OWASP, no corta, así que pasó inadvertida hasta esta revisión. Se fijó la 9.0.21 en los dos proyectos que traen SqlClient. Del lado del frontend había ocho, todas en dependencias de desarrollo (Vite, Vitest y Puppeteer, que no llegan al sitio publicado); se actualizaron dentro de sus rangos y `npm audit` da cero.

La carga no se midió con NBomber, y el perfilado de memoria no se hizo. La carga la mide `scripts/medir-carga.py` contra la API andando (`CARGA.md`), que tiene la ventaja de medir el sistema como lo usa un cliente y no en proceso. Del perfilado de memoria de la sección 3 no había ninguna corrida registrada —las métricas del ADR 0015 dejan ver la memoria, pero nadie la había medido con el motor trabajando—, y RNF-09 se daba por verificado con la medición de carga, que no mira la memoria. Ahora lo mide `scripts/medir-estabilidad.py`: el motor sobre el parque completo, evaluado decenas de veces seguidas, con la memoria de la API y las conexiones abiertas tomadas en cada vuelta (`ESTABILIDAD.md`).

ADR 0009 — Patrones de resiliencia: circuit breaker y bulkhead

Estado: Aprobado

Fecha: 2026-09-02

Origen: auditoría de arquitectura, `docs/trazabilidad/PROMPTS-CRITICOS.md` · PC-05

Contexto

La auditoría del diseño previa a la implementación (patrón B, abogado del diablo) encontró dos problemas que el diseño de los ADR 0002 a 0007 no resolvía.

El usuario espera a la IA. CU-006 registra la incidencia y CU-007 se dispara de forma sincrónica. Con el timeout de 15 segundos del ADR 0006, un solicitante puede esperar 15 segundos para que le confirmen que su pedido se registró. Peor: si el servicio está caído, **cada** incidencia paga esos 15 segundos. Veinte reportes seguidos en una mañana de corte de internet son cinco minutos de espera acumulada, todos terminando en el mismo respaldo previsible.

El timeout protege al sistema de colgarse, pero no lo protege de un servicio que está consistentemente caído. Es la diferencia entre fallar rápido una vez y fallar lento muchas veces.

Una evaluación larga puede dejar sin atender a los usuarios. El motor predictivo evaluando el parque completo abre conexiones sostenidas contra la base de negocio. Con el pool compartido, esa evaluación puede consumir las conexiones disponibles y las peticiones de los usuarios quedan esperando. Un proceso en segundo plano no debería poder degradar la operación interactiva.

Decisión

Se incorporan dos patrones de resiliencia, acotados a los dos puntos identificados. No se adopta una política general de resiliencia: cada patrón responde a un hallazgo concreto.

Circuit Breaker sobre `IProveedorIA`

Un disyuntor delante de la llamada al servicio externo, con tres estados:

- **Cerrado** — las llamadas pasan. Se cuentan los fallos consecutivos.

- **Abierto** — tras **5 fallos consecutivos**, el circuito abre y durante **60 segundos** las llamadas **no se intentan**: se va directo a la asignación de respaldo, sin esperar el timeout.
- **Semiabierto** — cumplido ese lapso, se deja pasar una llamada de prueba. Si responde, el circuito cierra; si falla, vuelve a abrir.

Cuenta como fallo lo mismo que ya define el ADR 0006: error, timeout, respuesta malformada, o respuesta que nombra un técnico inexistente.

La apertura y el cierre del circuito se asientan en bitácora, y el estado se muestra en la pantalla de Integración de IA: *"Sin respuesta desde hace 12 min · se está aplicando la regla de respaldo"*. Que el administrador vea esto es parte de la decisión: un sistema degradado que no lo dice deja al administrador sin motivo para revisar nada.

Bulkhead para los procesos en segundo plano

Las operaciones interactivas y los procesos en segundo plano —evaluación del parque completo, respaldos automáticos— usan **cadenas de conexión con pools separados** (`Max Pool Size` propio en cada una). Una evaluación larga puede agotar su propio pool; no puede tocar el de los usuarios.

Es la mitigación más barata posible para este hallazgo: no requiere infraestructura, sólo dos cadenas de conexión distintas hacia la misma base.

Alternativas consideradas

Opción	Por qué se descartó
Sólo timeout, sin circuit breaker	Es lo que ya define el ADR 0006 y es exactamente lo que la auditoría encontró insuficiente: protege de colgarse, no de fallar lento repetidamente.
Retry con backoff exponencial sobre la llamada de IA	Multiplica la latencia que es el problema a resolver. Se conserva sólo para la reevaluación en segundo plano, donde nadie está esperando.
Asignación asincrónica por cola	Elimina el problema de raíz, pero agrega una pieza de infraestructura desproporcionada para el alcance y rompe la expectativa de CU-007, que asigna al registrar.
Biblioteca de resiliencia completa (Polly)	Resuelve esto y mucho más, pero un disyuntor de tres estados son unas pocas decenas de líneas y acá conviene que el patrón esté escrito y sea señalable en la defensa, no delegado.

Consecuencias

Positivas

- Con el servicio caído, el registro de una incidencia responde en el tiempo normal en lugar de esperar el timeout. El sistema degrada de forma predecible.
- La degradación es **visible**: queda en bitácora y se muestra en pantalla.
- El disyuntor es fácil de demostrar: forzando fallos en el proveedor simulado se puede mostrar la transición de los tres estados. Se suma a CP-14 como caso de prueba propio.
- El bulkhead elimina una clase entera de incidentes —"el sistema se puso lento a las 3 de la mañana"— por el costo de una cadena de conexión adicional.

Negativas y riesgos

- **Con el circuito abierto se dejan de intentar llamadas que podrían funcionar.** Si el servicio se recupera al segundo 5, el sistema no lo aprovecha hasta el 60. Es el costo aceptado del patrón, y el estado semiabierto lo acota.
- Los umbrales —5 fallos, 60 segundos— son una elección inicial sin datos que la respalden. Quedan configurables y hay que revisarlos con el comportamiento real.
- El disyuntor es estado en memoria: con varias instancias de la API, cada una tiene el suyo y abren por separado. Para el alcance del TFI hay una sola instancia; con más de una habría que compartir el estado, y eso está fuera de alcance.
- Los pools separados dividen un recurso finito: mal dimensionados, el proceso en segundo plano se queda corto y tarda más. Es un intercambio deliberado —preferimos un job lento a usuarios esperando— pero hay que dimensionarlo con las pruebas de carga del ADR 0008.

Precisión de la implementación (02/09/2026)

El disyuntor es por organización, no uno solo para todo el sistema. Esta decisión no estaba en la versión original del ADR y apareció al implementarlo.

El motivo es de aislamiento y no de rendimiento. Los modos de falla del proveedor son en buena medida **por cliente**: la clave de API venció, la cuenta se quedó sin crédito, la organización configuró un modelo que ya no existe. Con un disyuntor compartido, cualquiera de esas situaciones en un cliente abriría el circuito para todos los demás, y organizaciones cuyo proveedor responde con normalidad pasarían a asignación de respaldo sin ninguna razón. Sería el mismo error de diseño que el ADR 0005 evita en el acceso a datos, trasladado a la resiliencia.

La implementación es un registro de disyuntores indexado por organización, con el mismo umbral y la misma espera para todos. Todo lo demás que dice este ADR —los tres estados, los

cinco fallos, los sesenta segundos, la única llamada de prueba en semiabierto— vale igual para cada disyuntor del registro.

La consecuencia negativa ya anotada se mantiene y se amplía: el estado sigue siendo de esta instancia de la API, y ahora hay un disyuntor por cada organización activa en lugar de uno. Son unas pocas decenas de bytes cada uno, así que el costo es irrelevante; lo que hay que tener presente es que el diagnóstico de «el circuito está abierto» pasa a ser una pregunta por organización y no del sistema.

ADR 0010 — Cifrado reversible, separado del hash de contraseñas

Estado: Aprobado

Fecha: 2026-09-02

Origen: auditoría contra el modelo de referencia de 3.º año (CU.Arq.006)

Contexto

El sistema tenía una sola primitiva criptográfica: PBKDF2-HMAC-SHA256 sobre las contraseñas, con 120 000 iteraciones y el formato `PBKDF2$iter$salt$hash` del proyecto de referencia. Alcanza para autenticar y no alcanza para nada más, porque un hash es de un solo sentido por definición.

El caso que lo dejó en evidencia fue la clave de API del proveedor de IA. La tabla `ConfiguracionAsignacion` tiene desde el principio una columna `api_key_cifrada`, y estaba **vacía**: la clave se leía de la variable de entorno `ANTHROPIC_API_KEY`, con este comentario en `Program.cs`:

> *La clave sale de la variable de entorno y no de la columna `api_key_cifrada`:*
>
> *todavía no hay servicio de cifrado, y guardar una clave en claro en la base*
>
> *sería peor que no guardarla.*

El comentario era correcto y la solución, provisoria. Tiene dos consecuencias que importan al producto, no sólo a la arquitectura:

1. **Una sola clave para todas las organizaciones.** El plan Partner vende atender a varias PyMEs; con la clave en el entorno, todas comparten la credencial de quien opera el servidor, y el consumo de una organización se factura contra la cuenta de otra.
2. **El cliente no puede administrar su propia credencial.** Cargar una clave exige acceso al servidor, que es exactamente lo que un servicio en la nube no le da a su cliente.

Además, CU.Arq.006 del modelo de referencia pide encriptación y desencriptación como caso de uso propio, distinto del hash.

Decisión

Se agrega `ICifradoService`, **separado** de `HashContrasena` y sin reemplazarlo.

Los dos mecanismos coexisten porque resuelven problemas distintos. Una contraseña no hace falta recuperarla: el sistema sólo necesita verificarla, y poder recuperarla sería una debilidad. Una credencial de un tercero sí hay que recuperarla, porque hay que enviarla en cada llamada. Usar hash para la clave de API sería inútil; usar cifrado para las contraseñas sería peor que inútil.

Algoritmo: AES-256 en modo GCM. GCM cifra y autentica en la misma operación: un texto alterado no se descifra a basura en silencio, falla, porque la etiqueta de autenticación no coincide. Con AES-CBC habría que sumar un HMAC aparte para obtener lo mismo, y omitirlo es el error clásico de esta clase de código.

Formato: `AESGCM$v1$nonce$etiqueta$cifrado`, las tres partes en Base64. Sigue la forma del hash de contraseñas para que un valor almacenado se reconozca de un vistazo, y lleva versión adelante para poder rotar el algoritmo sin tener que adivinar con qué se cifró cada fila.

Nonce aleatorio por operación. Repetir un nonce con la misma clave es la única forma de romper GCM sin romper AES. El efecto observable es que cifrar dos veces el mismo texto da resultados distintos. Si no fuera así, alguien con acceso a la base sabría que dos organizaciones cargaron la misma clave sin necesidad de leerla.

Sin clave configurada, el servicio se niega a cifrar. No guarda en claro «para no romper». Que el sistema quede sin credencial es recuperable; una credencial de un tercero en claro en la base, no.

La clave de cifrado es distinta de la de firma de tokens. Quien pueda firmar tokens no tiene por qué poder leer las credenciales de los clientes. `arrancar.ps1` genera las dos por separado, y ninguna se versiona.

Cómo queda la clave de API

- Se guarda cifrada en `ConfiguracionAsignacion.api_key_cifrada`, **por organización**.
- La clave guardada **gana** sobre la variable de entorno: si el cliente cargó la suya, es la que quiso usar. El entorno queda como respaldo para desarrollo.
- **No hay endpoint que la devuelva.** Se puede cargar y borrar. Lo que se consulta es de dónde sale la que está en uso —`BASE`, `ENTORNO` o `NINGUNA`—, porque eso cambia quién la controla, y nunca su valor.
- La bitácora asienta el hecho de que se cargó una clave y jamás el valor, ni recortado: los primeros caracteres de una credencial ya son información sobre ella.

- Si la clave de cifrado se rota sin volver a cargar las credenciales, el descifrado falla; se asienta el error y el sistema cae al proveedor simulado. Es preferible asignar con heurísticas y decirlo, a no asignar.

Consecuencias

A favor

- Cada organización administra su propia credencial, condición necesaria para que el plan Partner pueda para facturar el consumo donde corresponde.
- La credencial no viaja en texto plano ni en la base, ni en la bitácora, ni en ninguna respuesta de la API.
- El sistema falla ruidosamente ante un dato alterado, en vez de operar con basura.
- CU.Arq.006 queda cubierto con un caso de uso real y no con una demostración.

En contra

- Aparece una clave más para administrar en el despliegue, y perderla vuelve ilegibles las credenciales guardadas. Se mitiga documentándolo en la plantilla de configuración y generándola automáticamente en `arrancar.ps1`, pero el riesgo operativo es real y no se elimina.
- El cifrado es simétrico: quien tenga a la vez la base y la clave puede descifrar. Un esquema de sobres con una clave maestra externa —KMS— sería más fuerte, y queda fuera de alcance porque supone infraestructura de nube que el proyecto no tiene.
- No hay rotación automatizada. Rotar hoy es cambiar la clave y volver a cargar las credenciales a mano. Con una sola credencial por organización es aceptable; con más habría que escribir la migración.

Alternativas descartadas

Seguir con la variable de entorno. Es lo que había. No resuelve la multi-organización ni deja al cliente administrar su clave, y deja `api_key_cifrada` como una columna muerta que el DER promete y el sistema no cumple.

Windows DPAPI. Menos código, y ata el sistema a Windows y a la máquina: la base respaldada y restaurada en otro servidor dejaría de poder descifrarse. El proyecto se despliega en Windows hoy, pero nada del resto de la arquitectura lo exige.

AES-CBC + HMAC. Equivalente en fuerza si se implementa bien, y el «si se implementa bien» es justamente el problema: hay que encadenar cifrado y autenticación en el orden correcto y comparar el HMAC en tiempo constante. GCM lo resuelve en una operación.

Cifrar también las contraseñas, para tener un solo mecanismo. Sería una regresión de seguridad grave: volvería recuperable algo que no tiene por qué serlo. La uniformidad no es un valor cuando los problemas son distintos.

ADR 0011 — El motor predictivo se dispara solo

Estado: Aprobado

Fecha: 2026-09-02

Origen: prueba de aceptación sobre el sistema corriendo

Contexto

El motor predictivo estaba completo: las cinco estrategias del ADR 0007, el scoring ponderado, la generación de alertas y la pantalla de análisis. Pasaba sus pruebas y, consultado por la API, devolvía datos correctos —47 equipos evaluados, 5 reglas activas, ranking con scores y niveles—.

Y sin embargo **sólo corría cuando alguien apretaba «Evaluar»**. Una revisión de las llamadas lo confirmó: nada invocaba `EvaluarParque` ni `EvaluarEquipo` fuera del controlador. Ni el alta de una incidencia, ni el registro de un mantenimiento, ni ningún proceso programado.

Eso no cumple RF-11. Un sistema que calcula el riesgo cuando se lo consulta es un informe, no un sistema predictivo: la alerta llega cuando alguien ya fue a buscarla, que es justo cuando ya no hace falta que le avisen.

El defecto era invisible desde el código y desde las pruebas. Cada pieza estaba bien y la que faltaba no existía, así que no había nada que leer mal ni ninguna prueba que fallara. Se detectó recorriendo el sistema corriendo.

Decisión

El motor pasa a tener **dos disparadores**, porque las condiciones que evalúa aparecen de dos maneras distintas.

Por hecho — `ReevaluarPorEvento(idEquipo)`

Se dispara al registrar una incidencia y al registrar un mantenimiento. Son los dos momentos en que cambian los datos que las reglas miran: una incidencia nueva puede completar una recurrencia o cruzar un umbral de acumulación, y un mantenimiento puede resolver una alerta de mantenimiento vencido.

Tres propiedades deliberadas:

- **No exige patente.** Es una consecuencia del alta, no una consulta que alguien pidió. Exigir `ALERTA_VER` haría que un solicitante —que puede reportar pero no ver alertas— no dispare la reevaluación de su propio equipo.
- **No propaga errores.** Que el análisis falle no puede hacer fallar el registro de la falla que lo disparó. El error se asienta en bitácora y la operación sigue.
- **Sólo asienta si generó algo.** Una línea de bitácora por cada incidencia diciendo «no pasó nada» haría ilegible la pantalla de auditoría.

Por tiempo — `BarridoPredictivo`

Un `BackgroundService` que recorre las organizaciones activas cada 24 horas, configurable, y con 0 lo desactiva.

Hace falta además del disparador por hecho porque **hay reglas que se cumplen sin que pase nada**: un mantenimiento vence, una garantía se acerca a su fin, un equipo cumple años. Esas condiciones aparecen por el paso del tiempo, y sin barrido la alerta esperaría a la próxima incidencia de ese equipo —que puede ser justamente la falla que se quería prevenir—.

Dos decisiones sobre cómo corre:

- **Contexto de sistema acotado.** El barrido no tiene sesión HTTP de la cual sacar el usuario y la organización, así que se le construye un contexto por organización con **sólo la patente que necesita**. No es un contexto de superusuario: si mañana el barrido intentara hacer otra cosa le faltaría el permiso y fallaría, que es el comportamiento buscado. El usuario queda vacío: la bitácora tiene que decir que lo hizo el sistema y no atribuírselo a una persona que no estaba.
- **Una organización que falla no frena a las demás.** Sin reglas configuradas, por ejemplo, `EvaluarParque` lanza a propósito. Ese caso se asienta y el barrido sigue con la siguiente.

Consecuencias

A favor

- RF-11 se cumple de verdad: la alerta aparece cuando la condición se cumple.
- La demostración deja de depender de que alguien apriete un botón en el momento justo.
- El contexto de sistema, acotado por patente, sirve para cualquier otro proceso en segundo plano que venga después.

En contra

- El alta de una incidencia hace más trabajo. Se acotó reevaluando **un equipo** y no el parque, y sin propagar errores, pero el costo existe.
- El barrido compite por conexiones con la operación interactiva. Por eso espera antes del primer ciclo —al arrancar la API está aplicando scripts y atendiendo el primer login— y por eso el bulkhead del ADR 0009 sigue siendo necesario.
- El barrido corre dentro del proceso de la API. Con varias instancias, cada una haría su propio barrido. Hoy hay una sola; el día que haya más va a hacer falta un candado, y conviene decirlo ahora y no descubrirlo con alertas duplicadas.

Alternativas descartadas

Sólo el barrido programado. Más simple, y demora la alerta hasta 24 horas desde el hecho que la habilita. La tercera incidencia del mismo equipo en una semana es exactamente el momento en que hay que avisar.

Sólo el disparador por hecho. No cubre las reglas que dependen del calendario, que son tres de las cinco.

Un programador externo —el Programador de tareas de Windows llamando a un endpoint—. Menos código propio, y suma una pieza de infraestructura por fuera del sistema, que hay que instalar y documentar aparte, y que falla en silencio si alguien la desactiva. Un `BackgroundService` viaja con la aplicación.

ADR 0012 — La máquina de estados de la incidencia vive en la BLL

Estado: Aprobado

Fecha: 2026-09-03

Origen: auditoría de patentes declaradas sin uso

Contexto

La incidencia se podía registrar y reasignar, y nada más. El catálogo `EstadoIncidencia` tiene **siete estados** y sólo dos eran alcanzables: *Pendiente de asignación* y *Asignada*. Las columnas `diagnostico`, `solucion` y `fecha_resolucion` existían en la base, el detalle de la incidencia las devolvía, y **ninguna consulta del sistema las escribía**.

La patente `INCIDENCIA_ATENDER` estaba en la base y ningún endpoint la exigía. Apareció buscando en el código las patentes declaradas que nadie usa, con la misma técnica que ya había encontrado la bitácora sin eventos de negocio y el motor predictivo que no se disparaba solo.

El agujero no era menor. Sin fecha de resolución no hay tiempo de reparación, no hay cumplimiento del SLA de cuatro horas que el documento promete, y el tablero no puede calcular el tiempo medio. El caso de uso central del técnico —tomar una incidencia, diagnosticarla y resolverla— no existía.

Y era invisible: cada pieza estaba bien escrita, nada fallaba, ninguna prueba se ponía en rojo. Faltaba código, y lo que no existe no puede fallar.

Decisión

Se implementa el ciclo de vida completo, con la máquina de estados **en la capa de negocio**.

Las transiciones

```
Pendiente de asignación  ->  Asignada (lo hace la asignación) | Anulada
Asignada                 ->  En curso | Anulada
En curso                 ->  En espera de repuesto | Resuelta | Anulada
En espera de repuesto    ->  En curso | Anulada
Resuelta                 ->  Cerrada | En curso   (reapertura)
Cerrada, Anulada         ->  nada. Son finales.
```

En un solo lugar

La máquina vive en `IncidenciaBusiness` y en ningún otro lado. La pantalla **pregunta** qué transiciones hay —`GET /incidencias/{id}/transiciones`— y ofrece las que el negocio devuelve. Si la máquina viviera también en el frontend habría dos, y se contradirían la primera vez que cambie una regla.

Tres reglas que van con la decisión

Resolver exige diagnóstico y solución. Una incidencia cerrada sin decir qué se encontró y qué se hizo no le sirve al próximo técnico que vea el mismo equipo, ni al motor predictivo, que se alimenta del historial.

La fecha de resolución la escribe el paso a* Resuelta *y la limpia la reapertura. Es el dato del que sale el tiempo de reparación: una incidencia reabierta que conservara la fecha vieja produciría un tiempo de reparación falso, y falso hacia abajo, que es el peor sentido.

Sólo la atiende el técnico asignado, o quien pueda reasignar cualquier incidencia. Es la misma regla que ya regía la reasignación; tenerla distinta acá sería una puerta lateral. *Anular* es aparte: cancela el trabajo de otro, así que exige la patente de reasignar cualquiera.

Dónde termina la validación y dónde empieza la guía

Esta parte se decidió **después de un error propio**, y conviene que quede escrita porque es el tipo de cosa que se repite.

El primer diseño hacía que el negocio reportara «falta cargar el diagnóstico» como impedimento de la transición a *Resuelta*. El negocio mira lo **guardado**, así que el botón quedaba deshabilitado para siempre: para guardar el diagnóstico hacía falta una transición, y la transición pedía el diagnóstico guardado. Un interbloqueo, y no se detectó leyendo el código sino usando la pantalla.

El reparto correcto es otro:

- **El negocio valida.** `Atender` rechaza resolver sin los dos textos, y es la única garantía: vale contra la API, contra Swagger y contra cualquier cliente futuro.
- **El negocio informa sólo los impedimentos que el formulario no puede conocer** —si hay técnico asignado, y quién es—.
- **El formulario guía.** Deshabilita *Resuelta* mientras falte texto, evaluando lo **escrito**, porque los dos viajan en la misma llamada.

Consecuencias

A favor

- El caso de uso central del sistema existe, y con él el tiempo de reparación y el SLA medible.
- Siete estados alcanzables en lugar de dos.
- Resolver una incidencia reevalúa el riesgo del equipo, porque cambió su historial. No propaga errores, igual que el alta.
- El avance queda auditado: cuatro códigos de evento nuevos, con el tiempo de resolución escrito en la línea de bitácora.

En contra

- La tabla de transiciones es un diccionario en código. Un catálogo en base sería configurable, y también permitiría dejar el sistema en un estado sin salida por configuración. Se eligió el diccionario: las transiciones de una incidencia no son algo que cada organización deba poder cambiar.
- La reapertura de una incidencia resuelta pierde la fecha de resolución anterior. Es deliberado —la nueva resolución tiene que medirse desde el alta— pero significa que el sistema no guarda cuántas veces se reabrió. Si eso importa, hace falta una tabla de historial de estados.
- El técnico asignado es el único que puede atender, así que una licencia obliga a reasignar. Es correcto y es fricción real.

Alternativas descartadas

Un endpoint por transición (`/resolver`, `/cerrar`, `/anular`). Se lee mejor en la URL y esparce la máquina de estados por seis controladores. La primera regla nueva habría que agregarla en seis lugares.

Estado libre, validado sólo por el catálogo. Es lo que había de hecho, y permite pasar de *Pendiente* a *Cerrada* sin que nadie la atienda: el registro queda consistente y la realidad que describe, no.

Un motor de flujo de trabajo configurable. Resuelve un problema que este sistema no tiene. Siete estados y ocho transiciones no justifican un motor, y sumarlo haría que la regla más simple del sistema fuera la más difícil de leer.

ADR 0013 — Reportes en PDF generados en el servidor

Estado: Aprobado

Fecha: 2026-09-03

Origen: RF-14, y la patente `REPORTE_GENERAR` declarada sin uso

Contexto

RF-14 pide reportes **filtrados y exportables**. La patente `REPORTE_GENERAR` estaba en la base, concedida a la familia `OPERACION_TECNICA`, y ningún endpoint la exigía: el requisito estaba en el documento y no en el sistema.

El formato lo eligió el autor: **PDF**. Es la decisión que condiciona todo lo demás, porque un PDF hay que componerlo y no sólo serializarlo.

Decisión

Se genera el PDF **en el servidor**, con **QuestPDF** bajo licencia Community.

Por qué en el servidor y no en el navegador

Tres razones, en orden de peso:

1. **El reporte tiene que poder explicarse solo.** Lleva impresa la fecha de emisión, quién lo generó y los filtros aplicados. Eso sólo es creíble si lo escribe el sistema: un PDF armado en el navegador lo firma el navegador.
2. **La bitácora tiene que saber que salió.** Un reporte impreso circula fuera del sistema, y saber que salió de acá, cuándo y con qué recorte es la condición para poder explicarlo después. Generándolo en el cliente, el servidor no se enteraría.
3. **Los datos ya están del lado del servidor.** Mandarlos al navegador para que los componga es mover el parque completo para dibujar una tabla.

Por qué QuestPDF

- **No necesita binarios nativos ni un navegador headless**, así que funciona igual dentro del contenedor de la API. Es la razón que descartó a `wkhtmltopdf` y a las soluciones basadas en Chromium: habría que meter un navegador en la imagen y la imagen pasaría de 350 MB a más de un giga.

- Licencia **Community**, gratuita por debajo del millón de dólares de facturación anual. Para este proyecto es limpio, y queda dicho acá para que no haya que averiguarlo el día que se use en producción.
- Se descartó **iText** por su licencia AGPL, que obliga a liberar el sistema entero.

El generador no sabe qué reporta

`IReporteService.Listado<T>` recibe encabezado, filas y definición de columnas. Por eso los tres reportes —parque, incidencias y mantenimientos— son **tres llamadas y no tres clases**: lo que cambia entre ellos son los datos y los títulos, no el documento.

Los reportes leen los datos que ya tiene el sistema

El del parque sale del mismo listado que la pantalla de activos; el de incidencias, del mismo buscador. Si el reporte trajera sus propias consultas, la pantalla y el papel podrían decir cosas distintas sobre el mismo día, y no habría forma de saber cuál miente.

Detalles de composición que son decisiones

- **Los filtros van en el encabezado y no al pie.** Un reporte filtrado que no dice con qué filtros se hizo se puede malinterpretar, y quien lo lea impreso no tiene forma de averiguarlo.
- **La cabecera de la tabla se repite en cada página.** El parque son tres hojas, y las hojas dos y tres sin encabezado son columnas de datos sin nombre.
- **Apaisado automático con más de cinco columnas.** Siete columnas en A4 vertical salen ilegibles.
- **Un reporte vacío dice que está vacío.** Una hoja con el encabezado y nada debajo se lee como un error de generación.
- **Los números se alinean a la derecha.** No es cosmético: una columna de importes alineada a la izquierda obliga a leer cada fila entera para comparar dos valores.

El PDF no se guarda

Se genera y se entrega. Un reporte guardado envejece, y alguien termina abriendo el de la semana pasada creyendo que es el de hoy. Los filtros viajan por *query string* justamente para que el enlace sea reproducible: el mismo enlace da el mismo recorte, con los datos del momento en que se pide.

`IReporteBusiness` va aparte

No se suma a `IAdministracionBusiness`. Esa interfaz ya mezcla cinco temas y es la violación de segregación de interfaces que el proyecto tiene anotada como deuda propia. No se le agrega un sexto.

Consecuencias

A favor

- RF-14 queda cumplido con un entregable que se puede imprimir y mostrar.
- El reporte del parque cruza el riesgo de la última evaluación. Ese cruce es el que lo vuelve útil para decidir y no sólo para inventariar. El de incidencias trae el tiempo medio de resolución, que recién ahora se puede calcular porque la fecha de resolución existe (ADR 0012).
- Cada emisión queda en la bitácora con quién, cuándo y cuántas filas.

En contra

- Una dependencia más en el backend, con una licencia que hay que leer. Está documentada acá y no descubierta después.
- Generar el PDF ocupa el hilo de la petición. Con 52 equipos son milisegundos; con un parque de miles habría que pasarlo a un trabajo en segundo plano y entregar el archivo cuando esté. No se hace ahora porque sería resolver un problema que el sistema no tiene.
- Los tres reportes traen la tabla completa a memoria para filtrar. Es consistente con cómo trabaja el resto del sistema y es lo que el requisito de carga del ADR 0008 tiene que medir.

Alternativas descartadas

CSV. Sin dependencias y en cuarenta líneas. Lo descartó el autor, y con razón para lo que se va a mostrar: un CSV no lleva encabezado, no lleva los filtros impresos y no se ve como un entregable.

Excel con formato. Más presentable que el CSV y menos que el PDF, y suma una dependencia igual. El PDF además se imprime y se firma, que es el recorrido habitual de un informe de servicios.

HTML imprimible desde el navegador. Sin dependencias, y el resultado depende del navegador y de la configuración de impresión de cada máquina. Un entregable que se ve distinto en cada computadora no es un entregable.

ADR 0014 — Gestión de identidad propia, sin IAM externo

Estado: Aprobado

Fecha: 2026-09-03

Origen: H-46. La pregunta apareció al revisar los requisitos de la cátedra,

que mencionan Keycloak como opción

Contexto

El sistema resuelve la identidad por su cuenta: usuarios en `PredictIT_Servicio`, contraseñas derivadas con PBKDF2-HMAC-SHA256 a 120.000 iteraciones, sesión con JWT firmado, y permisos en el modelo de familias y patentes que viene del proyecto de referencia de la carrera.

La alternativa evaluada fue delegar todo eso en un gestor de identidad externo —Keycloak es el candidato natural: es libre, se despliega en un contenedor y trae OpenID Connect, federación, autenticación en dos pasos y una consola de administración que ya está escrita.

La pregunta importa porque **el modelo de permisos es lo que la cátedra evalúa**. No es una pieza de infraestructura cualquiera.

Decisión

No se incorpora un IAM externo. La identidad y la autorización quedan en el sistema.

El motivo principal no es técnico

El modelo de familias y patentes, con el composite anidado y su guarda de ciclos, **es contenido de la materia**. Delegarlo en Keycloak lo convertiría en un archivo de configuración: los permisos pasarían a ser roles y scopes de un producto de terceros, y el composite —que es el patrón que el trabajo tiene que mostrar aplicado— desaparecería del código.

Un trabajo final que resuelve la autorización configurando un producto no demuestra lo mismo que uno que la implementa. Y lo que se pide demostrar acá es lo segundo.

Lo que además pesa

- **Dos fuentes de verdad.** El sistema es multi-organización y el aislamiento se aplica en el acceso a datos a partir del contexto de sesión. Con un IAM externo, la organización de un usuario viviría en Keycloak y las organizaciones vivirían en la base de negocio: dos lugares que hay que mantener sincronizados, y cualquier diferencia entre ellos se resuelve por accidente.
- **Un contenedor más y una dependencia de arranque.** Hoy el sistema levanta con cuatro contenedores y un comando. Keycloak agrega un quinto, su propia base, y un orden de arranque: sin el IAM disponible, nadie entra. Para una PyME que despliega esto en un servidor propio, es una pieza más que puede fallar un lunes a la mañana.
- **La revocación inmediata se pierde o se complica.** El token de PredictIT no lleva las patentes: se consultan en cada pedido, así que quitarle un permiso a alguien tiene efecto en la llamada siguiente. Un token de OIDC lleva los claims adentro, y revocar antes del vencimiento exige introspección en cada pedido —que es lo mismo que hace hoy el sistema, con un salto de red más— o aceptar que el permiso viejo siga valiendo hasta que el token expire.

Lo que se pierde, y hay que decirlo

Esto no es gratis. Delegando en Keycloak se tendrían, sin escribir código:

- **Autenticación en dos pasos.** El sistema no la tiene y no está planificada. Es la ausencia más significativa de esta decisión.
- **Federación e inicio de sesión único.** Entrar con la cuenta corporativa de Google o de Microsoft. Para el segmento PyME es una función que se pide.
- **Recuperación de contraseña por correo**, con sus plantillas y sus tokens de un solo uso. Hoy el restablecimiento lo hace un administrador.
- **Políticas de contraseña configurables** por organización.
- **Auditoría de autenticación ya hecha**, con su consola.

De esa lista, la que más duele es la de dos pasos. La mitiga en parte el bloqueo por intentos —cinco fallidos y la cuenta se cierra— más el límite por dirección de origen que se agregó en H-45, pero mitigar no es lo mismo que tener.

Consecuencias

- El código de seguridad es propio, así que los defectos de seguridad también son propios. Es lo que hace obligatoria la revisión contra OWASP Top 10 de `docs/tecnico/REVISION-OWASP.md`: sin un producto de terceros que se ocupe, la revisión no la hace nadie más.
- Cambiar de opinión más adelante es caro pero no imposible. `ISeguridadService` concentra la verificación de credenciales y `IContextoSesion` la resolución de permisos: un adaptador contra un IAM externo entraría por ahí. Lo que no se puede migrar sin rediseño es el modelo de familias y patentes, porque no tiene equivalente directo en los roles y scopes de OIDC.
- La decisión se revisa si el sistema sale de la etapa académica. En un despliegue comercial real, la falta de dos pasos y de inicio de sesión único pasa a ser un problema de venta, no una decisión de diseño.

Alternativas descartadas

Keycloak completo. Descartada por el motivo principal de arriba.

Keycloak sólo para autenticar, permisos propios. Es la opción intermedia y la más tentadora: se gana dos pasos y federación, y el composite se queda en el sistema. Se descartó porque el costo de infraestructura —el contenedor, la base, el orden de arranque— se paga igual y completo, y a cambio se resuelve sólo la mitad del problema. Es la que habría que reconsiderar primero si el proyecto sale de la facultad.

ASP.NET Core Identity. Trae el manejo de usuarios y contraseñas de Microsoft, sin contenedor extra. Se descartó porque impone su propio esquema de tablas, y el DER de seguridad del capítulo 10 es parte de lo que se evalúa: no se puede reemplazar por el de otro.

ADR 0015 — Métricas en formato Prometheus, sin stack de observabilidad propio

Estado: Aprobado

Fecha: 2026-09-15

Origen: la pregunta de si el sistema podía tener observabilidad con Grafana

o Prometheus. Al buscar el motivo apareció uno concreto: el RNF-05 no tenía forma de verificarse

Contexto

El documento compromete diez requisitos no funcionales. Cuatro son medibles y tres tienen su medición:

Requisito	Cómo se verifica hoy
RNF-04 · percentil 95 por debajo de 2 s	`scripts/medir-carga.py`, medición puntual
RNF-09 · sin fugas de memoria bajo carga	perfilado de memoria del ADR 0008
RNF-10 · degradación controlada de la IA	CP-14 y CP-15, automatizados
RNF-05 · disponibilidad del 95 % del horario laboral	**nada**

El RNF-05 es el único requisito del trabajo que se afirma y no se puede comprobar. El sistema tiene `/api/salud` y los contenedores tienen healthcheck, pero nadie registra cuánto estuvo arriba: el 95 % es una promesa sin medición.

Hay además un activo construido que no se ve desde afuera. El ADR 0009 incorporó circuit breaker y bulkhead, y el sistema tiene rate limiter, timeout configurable y reintento en la bitácora. Todo eso **funciona y no deja rastro consultable**: si el disyuntor se abrió tres veces esta semana, no hay dónde verlo. Lo único visible es el estado actual del proveedor, en una pantalla.

Decisión

El sistema expone sus métricas en formato Prometheus. No incorpora Prometheus ni Grafana.

La aplicación expone, la infraestructura recolecta. Es el reparto habitual y acá además es el que corresponde al alcance: el despliegue en servidor está declarado fuera de alcance para esta

entrega, así que montar un stack de recolección sería instrumentar un sistema que corre en una notebook.

Qué se expone

Tres grupos, en `GET /api/metricas`:

1. **Lo que da la biblioteca**: duración y conteo de peticiones HTTP por ruta, método y código de respuesta, y los contadores del runtime de .NET —memoria administrada, colecciones de basura por generación, hilos del pool—. Cubre RNF-04 y RNF-09 de forma continua y no puntual.
2. **El tiempo en línea** (`predictit_tiempo_en_linea_segundos`), que es lo que hace verificable el RNF-05. Un contador de uptime más el registro externo del recolector permiten calcular la disponibilidad real del período.
3. **Los dos mecanismos del ADR 0009 que se pueden observar sin instrumentar el código de negocio**, que es lo que convierte esta decisión en algo más que plomería:

- `predictit_disyuntores_abiertos`, cuántos circuitos no están cerrados en este momento. Es un gauge y no un contador de aperturas porque el disyuntor no avisa cuando cambia de estado: se lo consulta. Con recolección cada quince segundos, las transiciones de la serie dan igual las aperturas y el tiempo que estuvo abierta cada una;
- `predictit_peticiones_limitadas_total`, lo que rechazó el limitador. Éste sí es un contador, porque el limitador tiene un punto donde avisar.

Lo que no se instrumenta, y por qué. Cuántas consultas resolvió el proveedor real y cuántas cayeron al respaldo ya está contado, en la bitácora, con un código de evento por caso —`ASIGNACION_IA`, `ASIGNACION_HEURISTICA`, `ASIGNACION_RESPALDO`—. Duplicarlo como métrica sería un segundo contador que puede contradecir al registro de auditoría, que es la fuente. La pantalla de Integración de IA ya lo muestra desde ahí.

Se usa una biblioteca para el formato y no se escribe a mano

`prometheus-net.AspNetCore`. Es deliberadamente lo contrario de lo que se decidió con el disyuntor —que se escribió a mano «para que el patrón esté a la vista»— y la diferencia es cuál de las dos cosas evalúa la materia. El circuit breaker es un patrón de diseño y tiene que leerse en el código; un exportador de métricas es plomería de formato, y la parte no trivial —la agregación de histogramas para calcular percentiles— se hace mal con facilidad.

El endpoint no es público

Las métricas dicen qué rutas existen, con qué frecuencia se usan y cuánto tardan. Es material de reconocimiento para quien esté buscando por dónde entrar.

El endpoint se habilita sólo si hay un token configurado (`Metricas:Token`) y exige ese token en la cabecera `X-Metricas-Token`. **Sin token configurado, el endpoint no existe**: es seguro por omisión, y quien quiera recolectar tiene que decidirlo explícitamente.

No se usa el JWT del sistema porque un recolector no es una persona: no tiene sesión, no rota credenciales y pedirle que se autentique como usuario obligaría a crear un usuario de servicio con permisos, que es peor.

Alternativas consideradas

Alternativa	Por qué no
Prometheus y Grafana en el `docker-compose`	Dos contenedores más sobre los 2,3 GB que ya pesa el clon limpio, que según H-53 tiene que arrancar en la notebook de la defensa. Y un tablero de Grafana es un archivo JSON generado que hay que versionar y mantener al día, con el mismo problema que tenía el documento antes de automatizarlo
Como perfil opcional del compose (`--profile observabilidad`)	Es la opción intermedia razonable y queda como trabajo posterior. No se descarta por mala sino por alcance: primero el sistema tiene que ser observable, y recién ahí tiene sentido decidir con qué mirarlo
Escribir el exportador a mano	El formato de texto de Prometheus es trivial, pero la agregación de histogramas para percentiles no. Reimplementarla para ahorrar una dependencia madura es exactamente el tipo de decisión que el ADR 0004 rechazó en el otro sentido
Dejarlo fuera de alcance y declararlo	Es honesto y era una opción real. Se descartó porque el RNF-05 quedaría como el único requisito sin forma de comprobarse, y eso en la defensa es una pregunta de un renglón sin buena respuesta

Consecuencias

A favor. El RNF-05 pasa a ser verificable. Los patrones de resiliencia del ADR 0009 dejan de ser sólo código probado y pasan a tener registro de cuándo actuaron. Y la aplicación queda lista para cualquier stack de observabilidad sin tocarla.

En contra, y hay que decirlo. Se suma una dependencia —`prometheus-net`, con lo que implica para el control de componentes vulnerables del ADR 0008, que ahora la incluye—. Y las métricas se pierden al reiniciar el contenedor: sin recolector que las guarde, el tiempo en línea vuelve a cero. Eso es inherente a la decisión de no incorporar el stack, no un descuido.

Lo que esta decisión no resuelve. Sin recolector no hay alertas ni histórico. El sistema puede decir «el disyuntor está abierto ahora» pero no «se abrió ocho veces el martes», salvo que alguien esté recolectando. Está declarado acá para que no se lea como una promesa de más.

0016 · Alta de usuarios con contraseña temporal y sesión sin patentes

Estado: Aprobado · 20/10/2026

Contexto

RF-02 y CU-002 prometen la gestión de usuarios desde el sistema, y el sistema sólo sabía modificar los que venían de los datos iniciales: **no había alta**. La pantalla decía que se hacía desde la base y CP-02 figuraba aprobado con pruebas de otra cosa.

Dar de alta un usuario obliga a resolver algo que las otras altas del sistema no tienen: **cómo llega la primera credencial a manos de su dueño**, y qué puede hacer esa cuenta en la ventana que va desde que se crea hasta que su dueño la controla. Es una ventana de riesgo real: quien la cree conoce la credencial, y si la cuenta ya tiene todos sus permisos, el administrador puede operar como ella sin dejar rastro distinguible.

Condiciona además un requerimiento no funcional: RNF-04 fija cómo se guardan y verifican las credenciales, y el alta es el único momento en que el sistema genera una.

Alternativas consideradas

	Correo de invitación con enlace	El administrador fija la contraseña	Contraseña temporal de un solo uso
Dependencias	Servicio de correo saliente, que el sistema **no tiene** y que el plan declara fuera de alcance	Ninguna	Ninguna
Ventana de riesgo	Nula: la credencial la elige el dueño	Permanente si el usuario no la cambia, y nada lo obliga	Acotada: la cuenta no sirve para nada hasta el cambio
Quién conoce la credencial	Sólo el dueño	El administrador, para siempre	El administrador, hasta el primer ingreso
Entrega fuera de banda	No hace falta	Hace falta	Hace falta
Trazabilidad del primer ingreso	El consumo del enlace	Ninguna: es un ingreso más	El asiento distingue el reemplazo de la temporal
Costo de revertir	Alto: arrastra infraestructura de correo	Bajo	Medio: una columna y la semántica del login

El correo de invitación es la mejor de las tres y **no es ejecutable**: exige una pieza que el producto declara fuera de alcance en 10.5.1.10.5. Elegirla habría sido prometer en un ADR lo mismo que el capítulo 6 tuvo que dejar de prometer (D-104).

Decisión

El alta la hace el Administrador desde el sistema y **genera una contraseña temporal** que se muestra **una sola vez**, en la respuesta del alta. No se guarda en claro ni se puede volver a consultar: se persiste su hash con el mismo PBKDF2 de 600.000 iteraciones que el resto.

La cuenta queda marcada con `debe_cambiar_contrasena`, y **mientras esa marca esté puesta la sesión se arma sin ninguna patente** (`SeguridadService.PatentesDeLaSesion` devuelve el conjunto vacío). Toda operación protegida responde 403 sin que ningún controlador tenga que saber de esto; lo único que queda disponible es lo que no pide patente: la sesión, el cierre y el cambio de contraseña. **La pantalla que obliga a cambiarla es la comodidad; la garantía está en el servidor.**

El alta **exige las dos patentes**, `USUARIO_GESTIONAR` y `ROL_GESTIONAR`, y no una: dar de alta es también asignar roles, y con una sola alguien podría crearse un administrador.

El cambio de contraseña propia —`POST /api/auth/contrasena`— **pide sesión y no patente**, porque quien está obligado a cambiarla todavía no tiene ninguna, y **comparte el límite por origen del login**: verificar la contraseña actual es otra puerta para probar contraseñas.

Lo que se pierde, y hay que decirlo

La contraseña temporal viaja fuera del sistema. Alguien tiene que dictársela, mandarla por chat o escribirla en un papel, y el sistema no sabe ni puede saber por dónde fue. Es el precio exacto de no tener correo saliente, y no se disimula: es el primer motivo para incorporarlo.

El administrador conoce la credencial inicial. La ventana está acotada por la marca, no cerrada: entre el alta y el primer ingreso, quien dio de alta puede entrar como ese usuario. No podría hacer nada —la sesión no tiene patentes— pero podría cambiar la contraseña y quedarse con la cuenta. Lo que sí queda es el rastro: el asiento del reemplazo de la temporal dice quién lo hizo y cuándo, y una cuenta reclamada por su dueño no se reclama dos veces.

Consecuencias

Positivas

- No agrega infraestructura: el alta funciona con lo que el sistema ya tiene.
- La ventana de riesgo se cierra del lado del servidor y no del formulario, así que no depende de que el frontend se comporte.
- El primer ingreso es distinguible en la bitácora de cualquier otro.
- CP-02 pasa a describir una operación que existe, y la cubren dos pruebas de extremo a extremo: `AltaDeUsuarioPorLaApiTests`.

Negativas

- Una cuenta recién creada cuyo dueño nunca entra queda inservible y ocupando un usuario y un correo, que son únicos. No hay caducidad de la temporal.
- La columna `debe_cambiar_contrasena` mete una condición en el camino de login, que es el camino que más conviene mantener simple.

Riesgos asumidos

- Sin caducidad, una temporal entregada y olvidada sigue sirviendo. Se acepta porque la cuenta no tiene permisos hasta que se usa; cuando entre el correo saliente, la invitación con enlace vencible reemplaza a esta decisión.

Relación con el resto

- Amplía 10.3.3 y precisa 10.2.2 RF-02 · D-98
- Reemplaza, para el alta, la parte de 0014 que daba por sentado que los usuarios llegaban con los datos iniciales.

Fuentes

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

ISO/IEC. (2011). *Systems and software engineering. Systems and software Quality Requirements and Evaluation (SQuaRE). System and software quality models* (ISO/IEC 25010:2011).

Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Microsoft. (2024). *Documentación de ASP.NET Core y .NET 9*. Microsoft Learn. https://learn.microsoft.com/aspnet/core

Microsoft. (2024). *Documentación de SQL Server 2022*. Microsoft Learn. https://learn.microsoft.com/sql/sql-server

National Institute of Standards and Technology. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). https://pages.nist.gov/800-63-3/sp800-63b.html

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2.ª ed.). Pragmatic Bookshelf.

OWASP Foundation. (2021). *OWASP top 10: 2021*. https://owasp.org/Top10/